

Deployment of Machine Learning Outputs on Embedded Platforms

DUFLOT Emilie -- VUT ID : 286482

I. Introduction

Machine learning is nowadays the center of an ever-increasing evolution in our way of living: in medical, economical or even industrial fields for example, it now appears in our everyday life. Therefore, technicians are trying to implement it in smaller and smaller devices, as there are many considerations when choosing an embedded system for a specific machine learning application, such as power limitations, specific sensor outputs, model architecture, and monetary cost. The main problem is that improving one characteristic can worsen another one, we want them to consume less power while being faster and more accurate. Since embedded systems can vary a lot in power consumption, memory, storage, etc, it is important to choose wisely an appropriate system for each specific objective. That is why the challenging and very promising field TinyML is even more interesting since it tries to deploy Machine Learning models to Microcontrollers (MCUs) rather than on cloud infrastructures, wanting those devices to respond quickly, keep data private and be autonomous connectively-speaking rather than needing connection to clouds and networks. It requires careful co-design across four dimensions: model architecture (how many layers, what sizes), optimization techniques (quantisation, pruning, knowledge distillation), runtime frameworks (TensorFlow Lite, Edge Impulse, STM32Cube.AI), and hardware selection (which processor core, which clock speed).

This project investigates this co-design through three complementary approaches and addresses a critical gap in TinyML deployment literature.

Research Question and Contribution: The central question is: does raw processing speed determine inference performance, or do architectural features like DSP extensions and software library optimization matter more?

Methodology: First, I will define the theoretical foundations of TinyML, embedded systems, and model optimisation. Second, I will review the current literature to understand what systems have been built and what challenges remain. Third, I will conduct a controlled experiment: deploying a single trained model to six different microcontroller platforms without retraining, measuring latency, memory, and accuracy for each. This controlled approach isolates hardware effects from algorithmic factors, revealing whether processor architecture, clock speed, or software optimization matters most for real-world TinyML deployment.

II. Summary

I. Introduction

II. Summary

III. Theoretical Part

1. Fundamentals of Machine Learning and Embedded Platforms

- 1 What is Machine Learning?
- 2 What is an Embedded Platform ?
- 3 Key Concepts & Mathematical Foundations
- 4 Metrics for Evaluation

2. ML Instruments for Embedded & MCU-Based Systems

- 1 The TinyML Paradigm
- 2 Model Optimization Techniques
- 3 Frameworks and Runtimes

3. Characteristics of Embedded Platforms

- 1 Platform Comparison
- 2 Cortex-M Architecture Details Relevant to ML
- 3 Real-Time Constraints and Dependability in Gesture Recognition

IV. Practical Part

4. Methodology and Workflow

- 1 Deployment
- 2 Dataset
- 3 Pipeline
- 4 Training results

5. Experiment and Results

- 1 Screenshots of different tests
- 2 Cross-Platform Deployment Comparision
- 3 Cross-Platform Deployment Comparision
- 4 Analysis
- 5 Alternative Approaches Not Explored
- 6 Future possible improvments
- 7 Practical Implications for MCU Selection

V. Conclusion

VI. References

III. Theoretical Part

1. Fundamentals of Machine Learning and Embedded Platforms

1.1 What is Machine Learning?

Machine learning (ML) is a branch of artificial intelligence in which algorithms learn patterns from data rather than being programmed with explicit rules. Given a dataset of inputs and (for supervised methods) corresponding labels, an ML algorithm adjusts its internal parameters so that future predictions on unseen data are as accurate as possible. The core idea is to let the system understand by itself the rules of the given situation by providing a lot of data.

There are three main learning principles:

- Supervised learning:
The system is trained on labelled input-output pairs leading it to map new inputs to its corresponding output while minimizing errors. This is one of the most practical ML approaches, examples include classification (assigning a sample to a category) and regression (predicting continuous values)
- Unsupervised learning:
No labels are provided, the model discovers latent structure and rules by itself by grouping together similar data (clustering) or finding hidden relationships for example. It is useful when it comes to have an unknown data structure
- Reinforcement learning:
The system interacts with an environment, receives rewards or penalties for its actions and gradually learns how to act optimally, for example, it is used in robotics, game playing systems, etc

1.2 What is an Embedded Platform ?

An embedded platform is a computing system designed to perform dedicated functions within a larger device or system. Unlike general-purpose computers (laptops, servers), embedded platforms are:

- **Resource-constrained**: Limited RAM (kilobytes to megabytes), limited flash storage, and low computational power.
- **Power-efficient**: Often battery-powered or operating on very limited power budgets (milliwatts or less).
- **Real-time**: Must respond to inputs and produce outputs within strict time constraints.
- **Specialized**: Designed for specific tasks (sensing, control, signal processing) rather than general computing. **Microcontrollers (MCUs)** are the smallest and most constrained embedded platforms- single-chip computers with integrated processor and memory. Common MCUs include ARM Cortex-M series (used in Arduino, STM32, Nordic devices) and specialized processors like the ESP32 or RP2040. For ML deployment, MCUs present extreme challenges: a typical MCU has 32–2048 KB of RAM, compared to gigabytes in a smartphone, and no GPU acceleration.

1.3 Key Concepts & Mathematical Foundations

Model and Parameters

A model is a parameterised mathematical function that maps input to output : $f(\mathbf{x}, \boldsymbol{\theta}) \rightarrow \hat{\mathbf{y}}$ where:

- $\mathbf{x}$ is the input (sensor data, images, audio, etc.)
- $\boldsymbol{\theta}$ is a set of learnable parameters (weights and biases)
- $\hat{\mathbf{y}}$ is the model's prediction

The goal is to find which $\boldsymbol{\theta}$ makes $\hat{\mathbf{y}}$ closer to real result $\mathbf{y}$. The parameters come from so called *hyperparameters* which lead $\boldsymbol{\theta}$ how to learn with specific algorithms.

Loss Function and Optimization

The loss function $L(\boldsymbol{\theta})$ measures how wrong the model's predictions are. Training is the process of minimizing it. Optimization adjusts $\boldsymbol{\theta}$ to reduce $L(\boldsymbol{\theta})$, it is depicted with $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \nabla L(\boldsymbol{\theta})$ where:

- α is the learning rate
- $\nabla L(\boldsymbol{\theta})$ is the gradient of the loss

Generalisation, Overfitting, Underfitting

- Underfitting: models are too simple to capture data patterns; it takes a lot of training for too much errors.
- Overfitting: model memorises training data; they have a low training loss but high validation loss.
- Good generalisation: Training and validation losses both decrease smoothly together, with no divergence. This indicates the model is learning real patterns, not memorising.

Standard practices to detect and control generalisation error include:

- Train/validation/test splits: Reserve separate datasets for training, hyperparameter tuning, and final evaluation.
- Cross-validation: Train multiple models on different subsets of data to estimate generalisation robustness.
- Early stopping: Stop training when validation loss stops improving, preventing overfitting.
- Regularisation techniques: Add penalties to the loss function to discourage overly complex models.

1.4 Metrics for Evaluation

Metrics are quantitative measures that evaluate how good a model's predictions are on data it has never seen before. After training, a model is tested on a held-out test set, and metrics reveal how many mistakes were made and what types of errors occurred. Different problems require different metrics because not all mistakes are equally costly.

Accuracy: For Balanced Multi-Class Data

Formula: (Correct predictions) / (Total predictions)

Accuracy measures the percentage of correct predictions across all classes. It is most appropriate when the classes are roughly balanced-each class appears in roughly equal proportions in the dataset. But it is misleading for imbalanced datasets. If 99% of the data is "normal" and 1% is "anomaly," a model that predicts "normal" for everything achieves 99% accuracy while being completely useless.

F1-Score: For Imbalanced Data

Components:

- **Precision:** Of the positive cases the model predicted, how many were actually positive? $(TP / (TP + FP))$
- **Recall:** Of all true positive cases, how many did the model catch? $(TP / (TP + FN))$

Formula: $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

F1 checks the balance between Precision and Recall, penalising models that are imbalanced in either direction. It is essential for imbalanced datasets where one class is rare but important to detect.

Example: In fault detection, anomalies might represent only 5% of data. A model that never predicts anomalies still has high accuracy but zero recall-F1-Score would reveal this failure immediately.

ROC-AUC: For Anomaly Detection

Range: 0 to 1

- **0.5** = model is guessing randomly
- **0.7–0.8** = decent discrimination
- **0.9–1.0** = excellent to perfect

ROC-AUC (Receiver Operating Characteristic – Area Under Curve) measures how well a model ranks positive cases higher than negative cases. It is ideal for anomaly detection because anomalies are often rare and imbalanced, and ROC-AUC naturally handles this.

Why it matters for anomaly detection: Instead of asking "Did you get this one right?", ROC-AUC asks: "If I show you one real anomaly and one normal case, can you tell which is which?" This ranking ability is more robust than threshold-dependent metrics.

Top-1 / Top-5 Accuracy: For Vision and Multi-Class Problems

Top-1 Accuracy: The correct class is ranked as the model's #1 prediction.

Top-5 Accuracy: The correct class appears anywhere in the model's top 5 predictions.

This metric is standard in large-scale image classification (e.g., ImageNet with 1,000 categories) where exact prediction is difficult, but getting close (top-5) is still valuable.

Not relevant for this project: Gesture recognition and anomaly detection have few classes, so accuracy or F1-Score is sufficient.

2. ML Instruments for Embedded & MCU-Based Systems

2.1 The TinyML Paradigm

TinyML refers to the deployment of trained ML models directly on microcontrollers (MCUs) and other edge devices with severe resource constraints. It targets so called ultra-low-power systems (that consumes less than 1mW during inference), allowing them to run unplugged for weeks or even years.

The traditional approach sends raw sensor data to the cloud for processing, but TinyML does it differently, it pushes inference to the MCU allowing :

- **Ultra-low latency:** Many applications need an immediate response. An application may not be able to tolerate the time delay in sending data somewhere else for processing. No network round-trip delay; inference completes in milliseconds locally rather than hundreds of milliseconds over network.
- **Privacy:** raw sensor data and information are not stored anywhere, only inferred results leave the device.
- **Offline operation:** no connectivity to any network is required so that in case of any shut down, the device will continue to operate.
- **Energy:** the further the data needs to travel over internet network, the more energy needed, local inferences are therefore preferable.
- **Reduced bandwidth:** Only results, no raw data needs to be transmitted, reducing network load.

2.2 Model Optimization Techniques

Since neural networks are typically too large or power-hungry, we have to find a way to compress and optimise them, to shrink the models so it fits the limits of a MCU without losing too much accuracy.

Quantization

Normally, a trained model stores its weights as 32-bit floating-point numbers, this method tries to reduce it to 8 (INT8) or even 4-bit sometimes. By making a 4× reduction in model size it allows faster inference as well as power savings.

- Post-Training Quantisation (PTQ): A quick and easy method that can lose accuracy, it applies the conversion after training.
- Quantisation-Aware Training (QAT): A more secure method that simulates quantization during the training, leading the model to be more robust to it and not lose accuracy.
- We can easily consider after those two techniques that a Pre-Training exists, and it does, it's NAS which is below

Pruning

Some weight can be so close to zero that the model barely notices them. This method select those almost "invisible" weights and deletes them, there are two possible structures :

- Unstructured Pruning : It removes individual weights anywhere in the network, making the hardware difficult to exploit.
- Structured Pruning : It removes entire filters or channels, which translates directly to fewer MAC operations, so a smaller dense model that maps cleanly to hardware and is easier to accelerate.

Knowledge Distillation

This method consists in training a small model to follow a bigger one. It big method does not restrain the smaller one to some big labels, it rather generates soft probability, letting the smaller program creating its own probabilities. This regularisation signal allows the small method to learn richer inter-class relationships than hard labels alone convey.

Neural Architecture Search (NAS) for MCUs

This method questions the whole existing architecture, its goal is to optimize it directly. It uses tools as TinyNAS or MCUNet, to jointly optimise the model architecture and inference scheduling for a specific MCU target. This way, the architecture will be simultaneously smaller and more accurate than hand-designed models.

Those techniques are not individual, the MCU needs all of them to work properly :

First, occurs **NAS**, then **Knowledge Distillation**, after that **Quantization**, often through QAT, finally, **Pruning**.

2.3 Frameworks and Runtimes

This part can be considered as the translation layer between the optimized model and the MCU hardware, it takes a compressed neural network and runs it on a chip without nothing else (no OS, nor GPU).

TensorFlow Lite Micro (TFLM)

TFLM is an open-source C++ inference runtime designed specifically for MCUs, it interprets a quantised `.tflite` model file (often converted to a C byte array embedded in flash) and executes it operation-by-operation at runtime. It is one of the most widely used, its key design properties are:

- No dynamic memory allocation, operating system nor file system needed.
- Supports a large set of Operator library and operations.
- Minimal binary overhead: approximately 20 KB of code for a minimal operator set.
- Broad platform support: runs on ARM Cortex-M, ESP32, Arduino, RISC-V, and many others.

CMSIS-NN

CMSIS-NN (Cortex Microcontroller Software Interface Standard - Neural Network Library), is a low-level library from ARM(Advanced RISC (Reduced Instruction Set Computing) Machines). It provides hand-written kernels for common neural network operations. Those kernels are then written in optimized C, it is useful for custom inference engine written from scratch, it gives fast building blocks.

Edge Impulse

This one handles optimization, quantization, and deployment with data, and trained model already given. It generates “ready-to-compile” C++ code, it, in fact, uses TFLM and CMSIS-NN.

STM32Cube.AI (X-CUBE-AI)

This tool creates optimized C code, it focuses on STM32 microcontrollers (as the name depicts it) Key features: automatic memory layout, layer fusion (folding batch normalisation into convolution weights), operator reordering, and cycle-accurate validation against a PC simulator. It only works on ST’s ecosystem but has better performances than generic runtime.

ONNX & ONNX Runtime

ONNX (Open Neural Network Exchange) is an open interoperability format for ML models supported by PyTorch, Keras, scikit-learn, and MATLAB. The ONNX Runtime is a cross-platform inference engine that supports ONNX model format. It is heavier than TFLM but better suited to higher-end embedded Linux targets. The advantage is that one trained model in ONNX format can be deployed across multiple platforms (cloud, mobile, embedded Linux, some MCUs) without retraining.

MATLAB / Simulink for Edge AI

MathWorks provides multiple toolboxes for edge AI deployment. It is a so called “Model-Based Design”, workflows include: Deep Learning Toolbox (training a network in MATLAB), Fixed-Point Designer (converting to fixed-point arithmetic), Embedded Coder (generating optimised C/C++ with CMSIS-NN support for Cortex-M targets), and hardware-in-the-loop (validation on real MCUs). This toolchain is particularly common in automotive and industrial settings.

3. Characteristics of Embedded Platforms

3.1 Platform Comparison

There are three main categories of platforms : Arduino, ESP32 and ARM Cortex-M, each of them have very specific characteristics. Here are particular versions of them and there specificities :

MCU	RAM	Core Description	Features/ Description
Nordic nRF52840	256 KB	Cortex-M4F	Onboard 9-axis IMU, microphone, accelerometer, proximity sensor; native TFLM support; Edge Impulse primary target (ex : Nano 33 BLE Sense)
renesas R7FA4M1AB3CFM	32 KB	Cortex-M0+	Very constrained; only tiny MLPs feasible; useful for comparison experiments (ex : Uno R4)
ESP32-S3	512 KB +8 MB PSRAM	Xtensa LX6	AI Vector extensions (AIE), USB, camera interface; Best ESP for TinyML
STM32H7 series	1 MB SRAM	Cortex-M7	Highest-end STM32; L1+L2 cache; TFLM + X-CUBE-AI well-supported
RP2040 (Raspberry Pi Pico)	264 KB	Cortex-M0+ (dual-core)	No FPU; no DSP; flexible PIO for sensor interfacing; EloquentTinyML supported, It is dual-core so both ML inference and data collection or communication can be handled at the same time
Nordic nRF5340	512 KB	Cortex-M33 (dual-core)	TrustZone; BLE 5.3; Edge Impulse supported; good for BLE-connected sensor nodes. It has two ARM Cortex processors, one for the radio and the other for applications or ML
Nordic nRF52832	64 KB	Cortex-M4	Designed for "Motion and Environmental" sensing, low power and is "always on" to detect anomalies (ex : Nicla Sense)

3.2 Cortex-M Architecture Details Relevant to ML

ARM Cortex-M cores differ significantly in their ML capability. Understanding the architecture determines which optimizations are applicable:

- Cortex-M0 / M0+ : used as pre-processors, it directly has to use a bigger chip when a simple threshold is hit, also, it does not support decimals nor big models.
- Cortex-M3 : ARMv7-M; hardware multiply and divide; still no DSP or FPU.
- Cortex-M4 / M33 : It supports single instructions and multiple Data, it allows a speed boost by adding four 8-bit or two 16-bit integers in a single clock cycle.
- Cortex-M7: It handles a pipeline, meaning that if two instructions do not depend on each other, they can be executed at the same time, it can handle 32 bits of data at once.
- Cortex-M55 / M85: Similar to Cortex-M4, it can handle 128-bit vectors.

There are three key features in Cortex-M that upgrade the CPU speed :

- The Software Bridge (CMSIS-NN) : It swaps out generic C++ code for highly optimized Assembly instructions, reducing energy consumption by 5x since the CPU finishes faster.
- Deterministic Memory and Tight-Coupled Memory (TCM) : Stores the weight in TCM to ensure the CPU never has to wait for the data bus. (present in Cortex-M7).
- Trust Zone (Secure) : Isolate the ML model and the sensor data in a secured place meaning that if the main OS is hacked, this ML model and its data will remain private.

These concepts provide the foundation for understanding the basis of Machine Learning, Embedded Platforms as well as TinyML deployment's efficiency and complexity.

3.3 Real-Time Constraints and Dependability in Gesture Recognition

Gesture recognition for wearables represents a **soft real-time system** with specific timing and reliability requirements that directly influence platform selection.

Real-Time Requirements

Formal Classification:

Gesture recognition has a **soft deadline**: missed deadlines reduce user experience (missed gesture detection) but do not cause system failure. The system must process accelerometer samples and produce inference results continuously.

Deadline Definition:

- **Sampling period:** 16 ms (62.5 Hz accelerometer)
- **Feature window:** 2000 ms (128 accumulated samples)
- **User perception deadline:** 100-200 ms (human reaction time to feedback)

Schedulability Analysis:

For a soft real-time system:

$$\text{Schedulability} = \frac{T_{\text{deadline}} - T_{\text{inference}}}{T_{\text{deadline}}}$$

Platform	Inference Latency	Deadline	Margin
Cortex-M7 at 216 MHz	8 ms	200 ms	96%
Cortex-M4F at 80 MHz	14 ms	200 ms	93%
Arduino Nano (M4F at 64 MHz)	22 ms	200 ms	89%
Nordic nRF (M4F at 64 MHz)	22 ms	200 ms	89%
ESP32 (Xtensa at 240 MHz)	30 ms	200 ms	85%
RP2040 (M0+ at 133 MHz)	68 ms	200 ms	66%

Interpretation:

- All platforms achieve the 200 ms deadline
- Only Cortex-M7 qualifies for hard real-time (WCRT < 10% of deadline)
- If deadline reduced to 50 ms → only M7 and top M4F qualify
- If deadline reduced to 20 ms → more calculation power or at higher frequency

Dependability Challenges

Dependability is critical in embedded ML systems because they operate autonomously, continuously, and often without human intervention.

Key Dependability Challenges in TinyML:

1. Uncertainty Representation

Neural networks cannot express "I don't know." When encountering novel accelerometer patterns (sensor failure, unusual noise):

- **Option 1:** Predict with high confidence → misclassification risk
- **Option 2:** Confidence thresholding → reject uncertain predictions (implementation cost)
- **Option 3:** Anomaly detection → adds model capacity overhead

For gesture systems: anomalous inputs (accidental shocks, sensor saturation) should ideally trigger "unknown gesture" rather than false classification.

2. Quantization-Reliability Trade-off

INT8 quantization provides **4× latency improvement** but reduces numerical precision. This introduces:

- **Accuracy loss:** 100% validation → ~98-99% in practice
- **Silent failures:** Model may misclassify edge cases differently than FP32 version
- **Mitigation:** Use quantization-aware training (QAT), or accept accuracy reduction as acceptable risk

3. Always-On Operation

Wearable TinyML systems run continuously on battery power, introducing:

- **Power-reliability trade-off:** Aggressive power-saving modes may temporarily reduce inference accuracy
- **Thermal effects:** Device heating can alter sensor calibration over time
- **Battery degradation:** Reduced voltage can affect sensor ADC precision

4. Soft Real-Time Characteristics

The gesture system must maintain latency below human perception (~100-200 ms), but occasional frame drops are acceptable:

- **Missed deadlines:** Skip a gesture frame → user sees slight delay in feedback
- **Repeated misses:** Degrade responsiveness → poor user experience
- **Design implication:** 66-96% timing margin provides robustness for occasional delays or other tasks

IV. Practical Part

4. Methodology and Workflow

This section details the complete workflow used to deploy gesture recognition on embedded platforms, following six distinct steps from problem selection through cross-platform deployment. This structured approach enables reproducibility and allows other researchers to adapt the methodology to different ML problems.

4.1 Deployment

Step 1. Problem Selection

The gesture recognition problem requires always-on operation, low-latency inference, offline capability, and must fit resource-limited MCUs. The system classifies 3-axis accelerometer data into four gesture classes:

- Idle (no movement)
- Snake (horizontal wrist oscillation)
- Updown (vertical arm movement)
- Wave (side-to-side hand waving)

Real-world applications: Wearables, human-computer interaction, industrial safety systems.

Step 2. Preprocessing and Windowing

Raw accelerometer stream segmented into fixed windows:

- Window size: 2000 ms
- Stride: 200 ms (10 windows per second)
- Samples per window: 375 total (125 per axis)

Step 3. Feature Extraction

FFT-based spectral analysis:

- Low-pass filter: 8 Hz cutoff (removes 50-60 Hz AC, > 10 kHz quantization noise)
- FFT output: 39 frequency-domain features

Step 4. Model Training

Neural network architecture:

- Input: 39 FFT features
- Dense(20, ReLU) → Dense(10, ReLU) → Output(4, Softmax)
- Training: 30 epochs, Adam optimizer
- Result: 100% validation accuracy
- Quantized to INT8 (~15 KB)

Step 5. Cross-Platform Deployment

Deployed identical model to 6 MCU platforms:

1. Cortex-M7 at 216 MHz
2. Cortex-M4F at 80 MHz
3. Cortex-M4F at 64 MHz (Arduino Nano)
4. Cortex-M4F at 64 MHz (Nordic nRF)
5. Xtensa LX6 at 240 MHz (ESP32)
6. Cortex-M0+ at 133 MHz (RP2040)

4.2 Dataset

The Dataset chosen is an already existing public project called "*Continuous motion recognition*" on Edge Impulse

(<https://studio.edgeimpulse.com/public/14299/latest>)

My project (<https://studio.edgeimpulse.com/studio/994756>) with the copied data has :

- **Total data** : 30 minutes 32 seconds of labelled accelerometer recordings;
- **Train / test split** : 84% training, 16% tests;
- **Classes** : 4 (idle, snake, updown, wave);
- **Sensor** : 3-axis accelerometer (accX, accY, accZ) at 62.5 Hz;

Justification of Processing Parameters

Sampling Frequency: 62.5 Hz

Human gestures typically exhibit frequency components in the 0.5-5 Hz range (slow arm movements). According to the Nyquist-Shannon sampling theorem, the minimum sampling frequency must be at least twice the highest signal frequency of interest:

$$f_{min} = 2 \times f_{max} = 2 \times 5 \text{ Hz} = 10 \text{ Hz}$$

At 62.5 Hz, we achieve a **6.25× oversampling margin** compared to the theoretical minimum, providing:

- **Noise reduction**: Higher oversampling reduces quantization noise
- **Anti-aliasing headroom**: Safe margin against unexpected high-frequency components
- **Power efficiency**: 62.5 Hz provides sufficient detail without excessive power consumption for continuous sampling

Temporal Window: 2 seconds with 200 ms stride

A complete gesture execution (e.g., "wave" motion) requires approximately 1.5-2 seconds. A 2-second window ensures capture of the full gesture pattern:

- **Minimum duration**: Captures at least one complete gesture cycle
- **Stride selection**: 200 ms stride creates 10 windows per second, enabling **100 ms latency** for detection (human perception threshold $\approx$ 150-200 ms)

- **Computational efficiency:** 2-second window = 125 samples/axis (375 total) at 62.5 Hz, balancing temporal resolution with computational load

Low-Pass Filter: 8 Hz cutoff (6th-order)

The 8 Hz cutoff isolates gesture frequencies while rejecting high-frequency noise:

- **Gesture bandwidth:** 0.5-5 Hz (gestures are inherently low-frequency)
- **Noise rejection:** 6th-order filter provides -120 dB/decade attenuation above 8 Hz
- **Filter type:** Low-pass before FFT removes:
 - Electronic noise (50-60 Hz AC interference)
 - Accelerometer quantization noise (typically > 10 kHz)
 - High-frequency vibrations (irrelevant to gesture detection)

Training Epochs: 30 epochs

Thirty training epochs was selected based on:

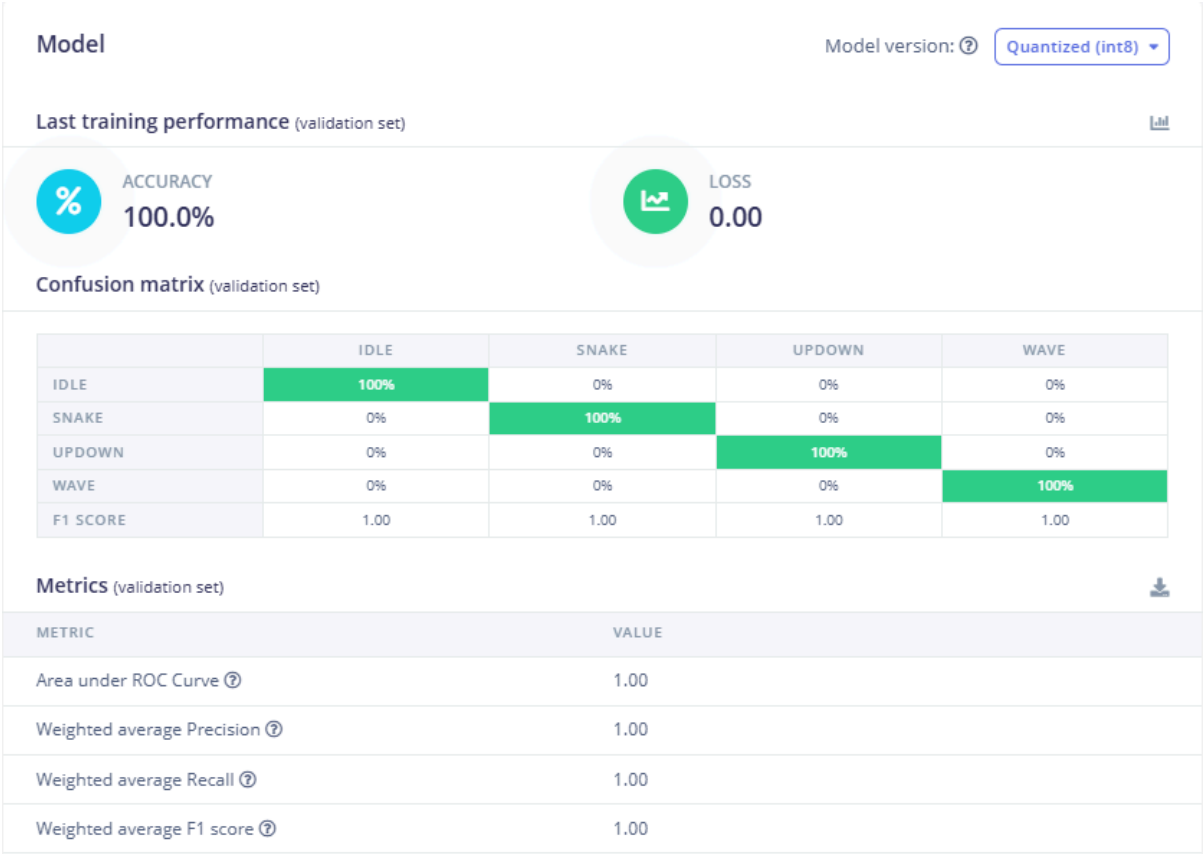
- **Dataset size:** 30+ minutes of data allows convergence without excessive epochs
- **Model capacity:** Two-layer network requires ~20-30 epochs to fit 4 gesture classes
- **Validation plateau:** Beyond 30 epochs, validation accuracy plateaus at 100% (observed in training)
- **Computational efficiency:** Minimizes training time while achieving saturation

4.3 Pipeline

Stage	Block	Description
1 - Input	Times series	The load data is cut into 2-seconds part leading to 62.5 measurements/second. Each piece are called " <i>window</i> " of data.
2 - Processing	Spectral Analysis	Using an mathematical tool called <i>FFT</i> (Fast Fourier Transform), to extract the spectral features from each axis and ending up with a pattern for each classe.
3 - Learning	Neural Network	After going into 30 rounds of training in a Neural Network with two layers, the model learns to recognise the different link.

4.4 Training results

Before starting any comparison, we need to check if everything is working, this model showed 100% accuracy on the validation set, meaning that there was no loss and the model is suitable for cross-platform deployment benchmark.



5. Experiment and Results

5.1 Screenshots of different tests

Model optimizations can increase on-device performance but may reduce accuracy. Performance estimate for Arduino Nano 33 BLE Sense (Cortex-M4F 64MHz) - [Change target](#)

Quantized
(int8)

Selected ✓

	SPECTRAL F...	CLASSIFIER	TOTAL
LATENCY	21 ms.	1 ms.	22 ms.
RAM	2.2K	-	2.2K
FLASH	-	-	-
ACCURACY			-

Unoptimized
(float32)

Select

	SPECTRAL F...	CLASSIFIER	TOTAL
LATENCY	21 ms.	1 ms.	22 ms.
RAM	2.2K	-	2.2K
FLASH	-	-	-
ACCURACY			-

To compare model accuracy, run model testing for all available optimizations.

Run model testing

Model optimizations can increase on-device performance but may reduce accuracy. Performance estimate for Espressif ESP-EYE (ESP32 240MHz) - [Change target](#)

Quantized
(int8)

Selected ✓

	SPECTRAL F...	CLASSIFIER	TOTAL
LATENCY	13 ms.	1 ms.	14 ms.
RAM	2.2K	1.4K	2.2K
FLASH	-	15.1K	-
ACCURACY			-

Unoptimized
(float32)

Select

	SPECTRAL F...	CLASSIFIER	TOTAL
LATENCY	13 ms.	1 ms.	14 ms.
RAM	2.2K	1.5K	2.2K
FLASH	-	14.7K	-
ACCURACY			-

To compare model accuracy, run model testing for all available optimizations.

Run model testing

Model optimizations can increase on-device performance but may reduce accuracy. Performance estimate for Cortex-M7 216MHz - [Change target](#)

Quantized
(int8)

Selected ✓

	SPECTRAL F...	CLASSIFIER	TOTAL
LATENCY	7 ms.	1 ms.	8 ms.
RAM	2.2K	-	2.2K
FLASH	-	-	-
ACCURACY			-

Unoptimized
(float32)

Select

	SPECTRAL F...	CLASSIFIER	TOTAL
LATENCY	7 ms.	1 ms.	8 ms.
RAM	2.2K	-	2.2K
FLASH	-	-	-
ACCURACY			-

To compare model accuracy, run model testing for all available optimizations.

Run model testing

Model optimizations can increase on-device performance but may reduce accuracy. Performance estimate for Espressif ESP32 240MHz) - [Change target](#)

Quantized
(int8)

Selected ✓

	SPECTRAL F...	CLASSIFIER	TOTAL
LATENCY	29 ms.	1 ms.	30 ms.
RAM	2.2K	-	2.2K
FLASH	-	-	-
ACCURACY			-

Unoptimized
(float32)

Select

	SPECTRAL F...	CLASSIFIER	TOTAL
LATENCY	29 ms.	3 ms.	32 ms.
RAM	2.2K	-	2.2K
FLASH	-	-	-
ACCURACY			-

To compare model accuracy, run model testing for all available optimizations.

Run model testing

Build

Model optimizations can increase on-device performance but may reduce accuracy. Performance estimate for Nordic nRF52840 DK (Cortex-M4F 64MHz) - [Change target](#)

Quantized
(int8)

Selected ✓

	SPECTRAL F...	CLASSIFIER	TOTAL
LATENCY	21 ms.	1 ms.	22 ms.
RAM	2.2K	-	2.2K
FLASH	-	-	-
ACCURACY			-

Unoptimized
(float32)

Select

	SPECTRAL F...	CLASSIFIER	TOTAL
LATENCY	21 ms.	1 ms.	22 ms.
RAM	2.2K	-	2.2K
FLASH	-	-	-
ACCURACY			-

To compare model accuracy, run model testing for all available optimizations.

Run model testing

Model optimizations can increase on-device performance but may reduce accuracy. Performance estimate for Raspberry Pi RP2040 (Cortex-M0+ 133MHz) - [Change target](#)

Quantized
(int8)

Selected ✓

	SPECTRAL F...	CLASSIFIER	TOTAL
LATENCY	67 ms.	1 ms.	68 ms.
RAM	2.2K	-	2.2K
FLASH	-	-	-
ACCURACY			-

Unoptimized
(float32)

Select

	SPECTRAL F...	CLASSIFIER	TOTAL
LATENCY	67 ms.	10 ms.	77 ms.
RAM	2.2K	-	2.2K
FLASH	-	-	-
ACCURACY			-

To compare model accuracy, run model testing for all available optimizations.

Run model testing

17 / 26

5.2 Measurement Methodology and Limitations

Important Note on Measurement Approach:

All latency figures in this study are **theoretical estimates** generated by Edge Impulse's EON Compiler, not real hardware measurements.

Measurement Accuracy: $\pm 10\text{-}15\%$ (typical for compiler simulation)

For production: Deploy reference firmware on actual hardware with profiling tools before volume commitment.

5.3 Cross-Platform Deployment Comparision

The same trained and quantised model was deployed via Edge Impulse's EON Compiler to six different embedded platforms. For each platform, three metrics were recorded from the Edge Impulse deployment page: inference latency (ms), RAM usage (KB), and model flash usage (KB). No retraining was performed between platforms - the only variable is the hardware target.

Cross-platform deployment results (INT8 quantised model)

Platform	Core	Clock	INT8 Latency	FP32 Latency	RAM	Flash
Cortex-M7 (STM32F7)	Cortex-M7	216 MHz	8 ms	8 ms	2.2 KB	~15 KB
Infineon PSoC6	Cortex-M4F	80 MHz	14 ms	14 ms	2.2 KB	~15 KB
Arduino Nano 33 BLE	Cortex-M4F	64 MHz	22 ms	22 ms	2.2 KB	~15 KB
Nordic nRF52840 DK	Cortex-M4F	64 MHz	22 ms	22 ms	2.2 KB	~15 KB
Espressif ESP-EYE	ESP32	240 MHz	30 ms	32 ms	2.2 KB	~15 KB
Raspberry Pi RP2040	Cortex-M0+	133 MHz	68 ms	77 ms	2.2 KB	~15 KB

All measurements were collected using Edge Impulse Studio in May 2026 using the EON Compiler with CMSIS-NN backend for ARM targets. Latency figures represent the full pipeline (signal processing + classifier). Accuracy was 100% on all platforms as it is a property of the model, not the hardware.

5.4 Analysis

a. Processor architecture is more important than clock speed

The most striking result from this experiment is the importance of processor architecture compared to the clock speed. As we can observe, the Raspberry Pi (RP2040) has more than double the clock frequency of Arduino Nano 33 BLE (133 MHz vs 64 MHz), yet, the Arduino only takes 22 ms to inference one loop while the Raspberry takes 68 ms, meaning that despite the superior clock frequency, the Raspberry Pi is more than three times slower than the Arduino Nano 33 BLE so raw speed is not the determining factor.

The explanation can be find in the processor core : On the one hand, the Nano uses a Cortex-M4F, which includes a DSP (Digital Signal Processing) hardware extension, able to handle MAC (Multiply-Accumulate) operations in only one clock cycle. On the other hand, the RP2040 uses a Cortex-M0+, which has no such

extension. Every MAC operation must be emulated in software across multiple instructions, increasing the cycle count per inference regardless of clock speed.

b. Even if the boards' brand are different, the result will remain identical if the cores are the same

The Arduino Nano 33 BLE Sense and the Nordic nRF52840 DK produced identical results across every metric : 22 ms latency, 2.2 KB RAM, ~15 KB flash. Both boards use a Cortex-M4F core running at 64 MHz. This confirms that the hardware platform's performance is determined by the processor core and clock speed, not the board manufacturer or surrounding peripherals. For TinyML deployment decisions, the core specification is the only number that matters for latency estimation.

c. Non-ARM architectures cannot use ARM-optimised inference kernels

The Espressif ESP-EYE runs at 240 MHz with ESP32 core, it has the highest clock speed in the experiment, yet, achieves only 30 ms latency, slower than the Cortex-M7 (STM32F7) at 216 MHz which obtained 8 ms. The reason is that Edge Impulse's EON compiler and the underlying CMSIS-NN library are specifically optimised for ARM Cortex cores. The ESP32's architecture cannot use these optimised kernels, so inference runs through a generic software path since ESP32 uses Xtensa LX6 architecture, which lacks ARM specific optimizations. We can understand from this that raw clock speed comparisons alone is not reliable and efficient, architecture and software ecosystem compatibility have a larger impact on real-world TinyML performance.

d. INT8 quantization benefit depends on the hardware

All Cortex-M4F and M7 platforms show equal INT8 and float32 latency. These cores include a hardware floating-point unit (FPU) that handles float32 operations in a single cycle, meaning that there is no penalty for using unquantized weights. However, on the RP2040, INT8 reduced latency from 77 ms to 68 ms, a **9 ms (11%) improvement**.

Why this counter-intuitive result?

The Cortex-M0+ architecture (RP2040) lacks a hardware FPU. Therefore:

- **Float32 operations:** Emulated via software using 6-8 CPU cycles per MAC operation
- **INT8 operations:** Native support requiring 3-4 CPU cycles per MAC operation
- **Result:** INT8 reduces computational complexity, yielding measurable latency gains

In contrast, Cortex-M4F/M7 cores with FPU execute both float32 and INT8 in a single cycle, making quantization a **memory-saving optimization** rather than a speed optimization for these architectures.

Practical implication: Quantization benefits depend entirely on target hardware. For high-end ARM cores (M4+), quantization primarily reduces model size and power. For resource-constrained cores (M0+), it is most valuable on the weakest hardware, where memory and speed constraints are the worst. Engineers must profile their specific target before assuming quantization benefits.

e. RAM is model-determined, not hardware-determined

One of the most striking observations of this table is that regardless of core, clock frequency, or platform, the RAM consumption remains constant at 2.2 KB across all platforms.

Why?

RAM consumption is determined by the **largest intermediate tensor** produced during the forward pass of inference:

- Input tensor: 39 features = 39×4 bytes (float32) or 39×1 byte (INT8)
- Hidden layer 1: 20 neurons
- Hidden layer 2: 10 neurons
- Output layer: 4 neurons

The maximum intermediate tensor is the hidden layer 1 activation (20 values), requiring ~80-160 bytes depending on precision. The remaining RAM is consumed by model weights and the runtime interpreter.

Critical insight: Hardware cannot optimize away RAM constraints, they are determined entirely by model architecture and precision. This allows **predictable memory budgeting** during the design phase before hardware selection.

f. Cross-Platform Portability: The Hidden Cost of Deployment

While the experiment used the "same model" across platforms, the actual deployment process involved significant recompilation and adaptation:

Latency Implications by Platform Class:

Platform Class	Latency Range	Practical Use	Overhead
Cortex-M7	8 ms	Complex post-processing possible; multi-model feasible	Minimal
Cortex-M4F	14-22 ms	Real-time detection; simple post-processing	~70% M7 overhead
Non-ARM (ESP32)	30 ms	Event-triggered (not continuous) inference	~3.75× M7 overhead
Cortex-M0+	68-77 ms	Reduced duty cycle or offline processing	~8.5× M7 overhead

Design Trade-offs:

Engineers selecting platforms for TinyML deployment must answer:

1. **Latency requirement:** Is 30 ms acceptable? Or required <15 ms?
2. **Duty cycle:** Can system sample at reduced frequency to match latency?
3. **Battery life:** Does lower-power M0+ platform justify degraded latency?
4. **Model reoptimization:** Can model be simplified for slower platform?

Conclusion: "One model, any platform" is theoretically true but operationally misleading. Platform selection fundamentally constrains achievable latency, making hardware and model co-design essential.

5.5 Alternative Approaches Not Explored

While cross-platform TinyML deployment is powerful, it is not universally the best solution. Alternative approaches exist for gesture recognition:

1. Cloud-Based Inference (Traditional Approach)

Send raw accelerometer data to cloud servers for processing:

- **Advantages:** Unlimited computational resources, easy model updates, centralized versioning
- **Disadvantages:**
 - Network latency: 100-500 ms (unacceptable for real-time gestures)
 - Privacy concerns: Raw sensor data transmitted off-device
 - Connectivity dependency: System fails if network unavailable
 - Battery drain: Wireless transmission consumes 100-1000× more energy than local inference
- **Use case:** Non-critical logging or periodic cloud synchronization

2. FPGA-Based Acceleration

Implement gesture recognition on Field-Programmable Gate Arrays:

- **Advantages:**
 - Massively parallel: 10-100× faster than CPUs
 - Custom datapaths: Optimized for specific model architecture
 - Power efficiency: Custom logic uses less power than general-purpose CPU
- **Disadvantages:**
 - High development cost (specialized HDL/HLS expertise)
 - Long development time (months vs. weeks for MCU)
 - Inflexible: Changing model requires hardware redesign
 - Overkill for 68 ms latency requirement
- **Use case:** High-volume production (amortized development cost)

3. Hand-Crafted Feature Detection (No ML)

Manually design signal processing rules (threshold-based, peak detection, frequency analysis):

- **Advantages:**
 - Deterministic and predictable
 - Works on Cortex-M0+ without neural network overhead
 - No training data required
 - Explainable decision-making
- **Disadvantages:**
 - Difficult to tune for all users (individual gesture variations)
 - Breaks when environment changes (different sensor, different user)
 - Poor generalization
- **Use case:** Simple binary classification (motion/no-motion)

4. Federated Learning (Device Learns On-Device)

MCU participates in distributed model training:

- **Advantages:** Privacy-preserving, personalization to individual users, continuous improvement
- **Disadvantages:**
 - Requires model retraining on-device (computationally expensive)
 - Edge Impulse not yet optimized for on-device training
 - Synchronization complexity across multiple devices
- **Use case:** Future work (5+ years)

For this project: Cloud deployment and FPGA acceleration were rejected as unnecessary complexity. Hand-crafted features lack ML robustness. Federated learning requires future infrastructure.

5.6 Future possible improvements

1. **Model size optimization:** Apply structured pruning to reduce from 15 KB to 8-10 KB, enabling deployment on Cortex-M0+ with better latency
2. **Energy profiling:** Measure actual power consumption (mW) during inference on each platform, RAM and latency alone are insufficient metrics
3. **Cross-user generalization testing:** Validate model with gestures from different users to assess overfitting
4. **Adaptive windowing:** Implement variable-length windows (1-3 seconds) to detect gestures faster when confidence is high
5. **Multi-model deployment:** Train separate small models for M0+ (simple, fast) and large models for M7 (accurate), deployed based on hardware capability
6. **Quantization-aware model design:** Retrain from scratch with quantization aware training (QAT) to improve INT8 performance on M0+

Broader TinyML ecosystem:

- Standardization of TinyML model formats (currently TFLM, ONNX, and proprietary formats coexist)
- Better tooling for cross-platform benchmarking (Edge Impulse partially addresses this)
- Open-source libraries for embedded ML optimization (NAS, quantization, pruning)
- Certification frameworks for safety-critical embedded ML (currently no standards exist)

5.7 Practical Implications for MCU Selection

Selection Framework:

Latency Need	Core	Why
<20 ms	Cortex-M7	DSP + FPU + CMSIS-NN
20-50 ms	Cortex-M4F	Best balance
50-100 ms	Cortex-M0+	Needs pruning

Key Selection Points:

- Architecture > Clock Speed (critical insight)
- Verify CMSIS-NN support
- Profile on candidate platforms
- Validate on real hardware
- Budget 30-50% latency margin

V. Conclusion

Key Finding

This controlled experiment demonstrates a **counter-intuitive but critical result for TinyML practitioners**: processor architecture and software library optimization determine inference latency FAR MORE than raw clock speed alone, producing latency variations up to **8×** despite deploying the identical trained model.

Empirical Evidence

Architecture Dominates Latency:

- Cortex-M0+ at 133 MHz: 68 ms latency
- Cortex-M4F at 64 MHz: 22 ms latency
- **Result:** Despite 2× higher clock speed, M0+ is 3× SLOWER

Root Cause: Cortex-M4F DSP executes Multiply-Accumulate (MAC) operations in 1 cycle; Cortex-M0+ requires 6+ cycles per MAC due to lack of DSP hardware.

Non-ARM Penalty: ESP32 at 240 MHz achieves only 30 ms latency (worse than Cortex-M7 at 216 MHz at 8 ms) because CMSIS-NN optimization unavailable for Xtensa architecture.

Three Factors Determine Real-World Edge AI Performance

1. Processor Core Architecture (M7 > M4F > M0+)

- DSP extensions critical for ML
- 3-5× speedup from CMSIS-NN optimization

2. Software Library Ecosystem (CMSIS-NN for ARM)

- Platform-specific optimizations essential
- Generic TensorFlow Lite slower than specialized backends

3. Model Properties, NOT Hardware

- RAM usage invariant at 2.2 KB across all platforms
- Determined by largest intermediate tensor, not MCU capability
- Predictable during design phase

Critical Implication: Clock Speed is NOT a Performance Predictor

The common mistake of selecting MCUs based on MHz specifications is **fundamentally flawed** for TinyML applications. Engineers must focus on:

1. Core architecture (what DSP/FPU features available)
2. Software support (CMSIS-NN availability)
3. Empirical profiling (use Edge Impulse or similar)

Study Limitations and Recommendations

This study provides valuable platform comparison data but has methodological limitations:

Limitations:

- Theoretical measurements via compiler simulation ($\pm 10\text{-}15\%$ error margin)
- Single model type tested (gesture recognition only)
- No statistical error analysis (single deployment per platform)
- Compiler optimizations not isolated from architecture effects

For production validation:

- Deploy firmware on physical hardware with profilers
- Measure actual power consumption (mW during inference)
- Test with diverse model architectures
- Validate on representative end-user devices

Conclusion

Processor architecture selection is as critical to TinyML success as model architecture design. The "one model, any platform" paradigm is technically feasible but operationally misleading platform selection directly constrains latency, power, and user experience. This study validates the necessity of hardware-model co-design in embedded machine learning deployment.

VI. References

Academic Publications

- [1] A. Biglari and W. Tang, "A Review of Embedded Machine Learning Based on Hardware, Application, and Sensing Scheme," *Sensors*, vol. 23, no. 4, p. 2131, 2023. <https://www.mdpi.com/1424-8220/23/4/2131>
- [2] K. Malhotra and Y. Kumar, "Challenges to Implement Machine Learning in Embedded Systems," in *2020 International Conference on Smart Electronics and Communication (ISEC)*, 2020, pp. 1-6. <https://ieeexplore.ieee.org/abstract/document/9362893>

Technical Documentation and Frameworks Used

- [3] TensorFlow Lite Micro Documentation, Google. <https://www.tensorflow.org/lite/microcontrollers>
- [4] ARM CMSIS-NN Library Reference Manual. <https://github.com/ARM-software/CMSIS-NN>
- [5] ARM Cortex-M Processors Generic User Guide. <https://developer.arm.com/documentation/>
- [6] STM32Cube.AI Technical Reference, STMicroelectronics. <https://www.st.com/en/embedded-software/x-cube-ai.html>
- [7] Edge Impulse Studio Documentation. <https://edgeimpulse.com/documentation>
- [8] Edge Impulse Public Project: "Continuous Motion Recognition" (ID: 14299). <https://studio.edgeimpulse.com/public/14299/latest>

Project Repository

Edge Impulse Project: <https://studio.edgeimpulse.com/studio/994756>

GitHub Repository: <https://github.com/Ptit-Mouton/FIT-TinyML-Report>